

Security Audit & Quality Review — Awareness

Newsletter Generator

Reviewer: DevSecOps multi-agent pipeline (parallel analyzers → manual line-level verification)
Target: Static, no-backend, vanilla-JS multi-page web app **Date:** 2026-05-30 **Audit type:** Current-state re-audit. The HIGH/MEDIUM vulnerabilities (H1, H2, M1, M2) were remediated in a first pass; a second pass then cleared the LOW/quality items (L1, L3, L4) and reviewed the flagged regexes (Q2). This report reflects the repository **as it stands now** and verifies every fix, then lists the remaining (deliberately deferred) architectural items.

Threat model: Each user supplies their own AI API key; all state is browser-local (localStorage / sessionStorage / IndexedDB); there is no server trust boundary. Realistic adversaries: (a) a compromised/spoofing RSS feed or CORS proxy injecting untrusted article HTML; (b) local disclosure of credentials the app promised to keep session-only.

Executive summary

Severity	Open	Remediated this session	Notes
Critical	0	0	none found
High	0	2 (H1, H2)	both verified fixed
Medium	1 (quality, deferred)	2 (M1, M2)	open item Q1 is architectural; Q2 reviewed → false positives, suppressed with justification
Low	1 (L2, deferred w/ reason)	3 (L1, L3, L4)	hardening + hygiene cleared
Code smell / anti-pattern	3 (architectural, deferred)	—	God objects, central sanitizer, 'unsafe-inline' CSP — need design, not a quick pass

Quality gate (verified after both passes): lint 0 errors / 157 warnings (was 291) · test:unit 164/164 · test:e2e 41/41.

Phase 1 — Findings

HIGH — remediated & verified (no longer exploitable)

H1 — Article-list XSS (RSS data → `innerHTML`) · `js/ui_controller.js` `renderArticles`

External RSS fields were interpolated raw into `innerHTML`. **Fixed:** `source/title/summary/type` and each `watchout` now pass through `escapeHtml()`; type-filter chips escape the type; the article link routes through a new `safeUrl()` that blocks `javascript:` / `vbscript:` / `data:` schemes. Locked by `tests/unit/render-articles-escaping.test.js`.

H2 — Microsoft Graph client secret persisted to disk · `js/ui_controller.js`, `js/db.js` The OAuth client secret was written to IndexedDB **and** `localStorage`. **Fixed:** stripped from both stores (`graphClientSecret: ''`), stored session-only under `awareness_graph_client_secret_session_v1`, restored from `sessionStorage` across all three load/apply paths, with a legacy-scrub on init. Documented in `docs/SECURITY.md`. Locked by a real-browser test in `tests/e2e/security-smoke.spec.js` (checks `localStorage` **and** IndexedDB) + a source-scan in `tests/unit/security.test.js`.

MEDIUM — remediated & verified

M1 — Keyword-chip self-XSS · `js/ui/sidebar_manager.js` Keyword text went raw into `innerHTML` + an injectable inline `onclick`. **Fixed:** installed a real `escapeHtml` (the prior `Utils.escapeHtml` fallback was a silent no-op — this also hardened feed-name rendering), escaped chip text, and replaced the inline handler with `data-keyword` + a delegated listener. Locked by `tests/unit/sidebar-keyword-escaping.test.js`.

M2 — Deploy-CLI argv command injection · `scripts/deploy-docker.mjs`, `scripts/deploy-server.mjs` `--name/--port/--host/--domain/--webroot` flowed unvalidated into shell strings. **Fixed:** new `scripts/lib/argv-validate.mjs` allowlist validators, wired in so a bad value fails fast before any interpolation. Locked by `tests/unit/deploy-argv-validation.test.js`.

MEDIUM — open (code-quality / latent)

Q1 — No centralized output-encoding (architectural) · 57 `innerHTML` assignment sites across `js/` Escaping is by-convention, applied per call site. H1 and M1 existed precisely because a few sites forgot to escape. 41 `no-unsanitized/property` + 5 `no-unsanitized/method` eslint warnings flag this surface. There is no shared sanitizer (e.g. DOMPurify) or trusted-types policy. **Risk:** the next new render site can silently reintroduce XSS.

Q2 — `detect-unsafe-regex` warnings (3 sites) — REVIEWED → false positives ·

`js/ai_summarizer.js:183`, `js/newsletter/core_templates.js:1737`, `js/ui_controller.js:860` Each was manually assessed for catastrophic backtracking: (1) the phishing-phrase matcher is a flat literal-phrase *alternation* with no nested quantifiers; (2) the stat extractor's `(?:[,\\.\d{3}])*` consumes mandatory progress each step (no overlapping unbounded quantifiers); (3) the doctype test is `^`-anchored with a single optional group. **None can ReDoS.** Rewriting behavior-critical regexes to satisfy a heuristic linter would risk the stat-extraction / phishing-detection / doctype-handling logic, so each is annotated with a justified `eslint-disable-next-line security/detect-unsafe-regex` instead. The 3 warnings are cleared.

LOW

ID	Finding	Status
L1	Docker nginx config missing the <code>baseline-critical-path-audit-results.json</code> deny rule	FIXED — rule added to <code>deploy/nginx.docker.conf</code>
L3	242 <code>no-unused-vars</code> warnings (mostly empty <code>catch (e)</code>) drowning real warnings	FIXED — eslint <code>no-unused-vars</code> set to <code>caughtErrors: "none"</code> + dropped an unused <code>runLocal</code> import; total warnings 291 → 157
L4	Leftover no-assertion temp test <code>tests/e2e/_tmp_eml.spec.js</code>	FIXED — removed (it asserted nothing; only <code>console.log</code>)
L2	<code>Access-Control-Allow-Origin: *</code> on dev servers (<code>dev_servers.mjs</code> , <code>ensemble_log_server.mjs</code> , <code>graph_relay_server.mjs</code>)	Deferred (by design) — the wildcard enables the legitimate cross-port dev POST (<code>:4173</code> app → <code>:4175</code> log/relay server); the servers bind to <code>127.0.0.1</code> only, so the practical risk is nil and tightening it risks breaking local dev tooling.

Code smells / architectural anti-patterns

- **God objects.** `js/ui_controller.js` (2,853 LOC), `js/newsletter/core_templates.js` (2,542), `js/ai_summarizer.js` (2,409), `js/editor.js` (1,887). High change-risk; H2's credential bug lived in `ui_controller.js` because secret-handling is interleaved with rendering, config, and delivery.
- **Duplicated `escapeHtml`.** Three independent implementations (`ui_controller.js`, the `sidebar_manager.js` fallback, `newsletter_builder.js`) — DRY violation. A referenced `App.Utls.escapeHtml` was never defined (latent no-op, now fixed in the sidebar but the missing canonical helper remains).
- **'unsafe-inline' CSP** driven by ~94 inline event handlers (documented in `docs/SECURITY.md`). This is the single biggest weakening of the app's XSS defense.
- **Source-scan tests as the lock for DOM-coupled fixes** (H1/M1) — see test-coverage gaps below.
- **Verified non-issues** (accepted/sound): `safeJoin()` traversal guard, dev-server filename validation, AI key + SMTP password session-only handling, ensemble-logger `isLocalhost()` gate, build-dist exclusions, editor `postMessage` origin checks.

Phase 2 — Triage: top 3 architectural risks to fix next

1. **No central sanitizer for 57 `innerHTML` sinks.** The codebase relies on every author remembering to escape. This is the root cause behind both XSS findings. *Recommendation:* introduce one canonical `App.Utils.escapeHtml` + a thin `setSafeHTML()` wrapper (or DOMPurify), and migrate the 46 flagged sites; add a lint rule upgrade from warn→error once migrated.
2. **`ui_controller.js` God object (2,853 LOC).** Config, secrets, article rendering, drafts, and delivery all share one mutable `state`. This is why a credential bug (H2) and a render bug (H1) coexisted in one file. *Recommendation:* continue the existing `js/ui/*` extraction — pull credential/SMTP handling into its own module with a narrow, testable surface.
3. **'unsafe-inline' CSP from ~94 inline handlers.** Until removed, any future escaping miss is directly exploitable (the CSP can't backstop it). *Recommendation:* migrate inline `onclick / onchange` to delegated `addEventListener` (the M1 fix is a template for this), then drop 'unsafe-inline' from `script-src`.

Test-coverage gaps — what was left untested and why

The suite is **33 files** (24 unit, 9 e2e); the gate is fully green. Coverage is deliberately uneven; the gaps below are by-design or structural, not oversights.

Source files with no dedicated test

File	Why it's untested	Indirect coverage
<code>js/responsive_layout.js</code>	Pure viewport-driven CSS-class toggling; almost no branching logic and no pure-function seam to assert.	Runs on every e2e page load (no assertions).
<code>js/ux_contract.js</code>	Injects shared chrome/menu/flow-stepper and route guards directly into the DOM; behavior is environmental, not a pure function.	Exercised by every <code>security-smoke</code> page-load (asserts pages load with no console errors), but its route-guard logic is not directly asserted — a real gap worth a targeted e2e.

Categorically untested by design (live integrations)

Per `AGENTS.md`, deterministic tests must pass with no network/keys, so these paths are **intentionally** excluded from `test:unit` / `test:e2e` and belong to `npm run audit:baseline` (treated as *blocked*, not *fail*, on outage):

- **RSS fetching over public proxies** (`js/rss_fetcher.js` network path) — e2e uses fixtures/route-mocks instead.
- **AI API calls** (`js/ai_summarizer.js` , `js/ai/*` network path) — deterministic tests use the local rules-based fallback engine.
- **Real SMTP / Microsoft Graph send** (`js/delivery_helpers.js` + relay) — only payload construction is unit-tested; actual delivery is never exercised in CI (no live relay/credentials).

Lighter-than-ideal coverage on the just-applied fixes (disclosed)

- **H1 (article XSS)** and **M1 (keyword XSS)** are locked by **source-scan** unit tests (asserting `escapeHtml / safeUrl` is present) plus e2e page-load smoke — **not** a runtime test that injects `<img onerror>` and asserts non-execution. Reason: `renderArticles` and the sidebar chip renderer are DOM-coupled with no pure-function seam, so a scan is the deterministic option. *Follow-up:* a small e2e that feeds a malicious fixture article and asserts no script runs would upgrade this to behavioral.
- **H2 (Graph secret)** has the strongest coverage: a real-browser e2e asserting absence from both `localStorage` and `IndexedDB`.
- **M2 (deploy argv)** has true behavioral unit tests on the pure validators + a manual dry-run check.

Broadly indirect-only

The large modules (`ui_controller.js` , `editor.js` , `ai_summarizer.js` , the `js/ui/*` siblings) are covered by **seam-contract** tests (`tests/unit/seam-contracts.test.js` , `app-modules.test.js`) for their module boundaries and by e2e for end-to-end flows, but the **majority of their internal functions have no isolated unit test** — a consequence of the God-object structure (Triage #2). Reducing file size would unlock finer-grained tests.

Verification appendix (after both remediation passes)

- `npm run lint` → **0 errors, 157 warnings** (down from 291; warnings do not fail eslint).
- `npm run test:unit` → **164 passed / 0 failed**.
- `npm run test:e2e` → **41 passed** (Chromium; was 42 before the no-assertion temp test L4 was removed).
- `npm run audit:baseline` not run here — it touches live RSS/AI endpoints; per `AGENTS.md` an outage there is *blocked*, not a regression signal.

Generated by the DevSecOps review pipeline; all line references verified against source on 2026-05-30.